

E-Commerce API — Frontend Integration Guide

Base URLs

- Local: `https://localhost:7236`
- Live: `https://ahmedkhalid25-001-site1.etrmpurl.com`

Swagger: `{baseUrl}/swagger`

All endpoints return JSON. All query params and JSON property names are **camelCase**.

1. Authentication

JWT Bearer tokens, expire in **1 hour**. Send as Authorization: Bearer {token}.

POST /api/Authentication/register — Public

// Request

```
{
  "email": "user@example.com",
  "password": "Ahmed@123",
  "displayName": "Ahmed Khaled",
  "phoneNumber": "01012345678", // optional
  "address": { // OPTIONAL — can be added later via PUT /address
    "firstName": "Ahmed",
    "lastName": "Khaled",
    "street": "Street 1",
    "city": "Alex",
    "country": "Egypt"
  }
}
```

POST /api/Authentication/login — Public

// Request

```
{ "email": "user@example.com", "password": "Ahmed@123" }
```

Both return the same UserDTO shape:

```
{  
  "id": "b4fe71f8-43a4-439d-a5aa-eb6accd677d5",  
  "email": "user@example.com",  
  "token": "eyJhbGciOi...",  
  "displayName": "Ahmed Khaled",  
  "phoneNumber": "01012345678",  
  "userName": "user@example.com",  
  "roles": ["Admin"],  
  "address": null // or the AddressDTO object if set  
}
```

GET /api/Authentication/CurrentUser — Auth required

Returns the **same UserDTO shape** (with a refreshed token). Use this on app startup to restore auth state and populate the profile page in one call.

GET /api/Authentication/emailexists?email= — Public

Returns true / false.

GET /api/Authentication/address — Auth required

Returns AddressDTO or 404 if not set.

PUT /api/Authentication/address — Auth required

// Request — same shape, creates or updates

```
{  
  "firstName": "Ahmed",  
  "lastName": "Khaled",  
  "street": "Street 1",  
  "city": "Alex",
```

```
"country": "Egypt"
}
```

Forgot Password — 3-step flow

Step 1 — POST /api/Authentication/forgot-password (Public)

```
{ "email": "user@example.com" }
```

Sends a 6-digit OTP by email. Valid 10 minutes.

Step 2 — POST /api/Authentication/verify-reset-code (Public)

```
{ "email": "user@example.com", "code": "123456" }
```

Response:

```
{ "message": "Code verified successfully.", "resetToken": "a3f8c91b4e2d4f7a..." }
```

 **Important:** the OTP is consumed here (one-time use). Store resetToken — this is what you send to Step 3, **not** the 6-digit code. The user only types the OTP once.

Step 3 — POST /api/Authentication/reset-password (Public)

```
{
  "email": "user@example.com",
  "resetToken": "a3f8c91b4e2d4f7a...",
  "newPassword": "NewPass@123"
}
```

2. Products

GET /api/Products — Public

Query params (all optional, all camelCase):

Param	Type	Notes
brandId	int	
subCategoryId	int	was typeId

Param	Type	Notes
categoryId	int	filters via product.subCategory.categoryId
search	string	partial match on name
sort	string	NameAsc NameDesc PriceAsc PriceDesc
pageNumber	int	default 1
pageSize	int	default 5, max 10

Response: PaginatedResult<ProductDTO>

```
{
  "pageIndex": 1,
  "pageSize": 5,
  "count": 41,
  "data": [
    {
      "id": 6,
      "name": "Xiaomi Redmi Pad",
      "description": "...",
      "pictureUrl": "https://.../Images/Products/XiaomiRedmiPad.jpg",
      "price": 6999,
      "discount": 0,
      "stockQuantity": 50,
      "inStock": true,
      "productBrand": "Xiaomi",
      "subCategory": "Laptops & Computers",
      "categoryId": 1,
      "categoryName": "Electronics & Mobiles"
```

```
}  
]  
}
```

pictureUrl is a full absolute URL — use directly in .

GET /api/Products/{id} — Public

Same shape as one item above.

POST /api/Products — Admin

```
{  
  "name": "New Product",  
  "description": "...",  
  "pictureUrl": "/Images/Products/placeholder.png",  
  "price": 99.99,  
  "discount": 10,  
  "stockQuantity": 50,  
  "brandId": 1,  
  "subCategoryId": 1  
}
```

PUT /api/Products/{id} — Admin

Same body shape.

DELETE /api/Products/{id} — Admin

3. Brands

Method	URL	Auth
GET	/api/Brands	Public
GET	/api/Brands/{id}	Public

Method	URL	Auth
POST	/api/Brands	Admin
PUT	/api/Brands/{id}	Admin
DELETE	/api/Brands/{id}	Admin

```
{ "id": 1, "name": "Samsung" }
```

Body for POST/PUT: { "name": "Brand Name" }

No direct Brand↔Category/SubCategory relation exists in the schema. A brand (e.g. Samsung) can have products under many unrelated categories. To get the brands relevant to a category/subcategory page, use the derived endpoints below — don't assume a static relation.

4. Categories & SubCategories

Schema: Category (1) → (many) SubCategory → (many) Product. Brand is fully independent.

GET /api/Categories — Public

```
[
  {
    "id": 1,
    "name": "Electronics & Mobiles",
    "description": "Smartphones, laptops, and tech accessories.",
    "subCategories": [
      { "id": 1, "name": "Smartphones", "categoryId": 1, "categoryName": "Electronics & Mobiles" },
      { "id": 2, "name": "Laptops & Computers", "categoryId": 1, "categoryName": "Electronics & Mobiles" }
    ]
  }
]
```

GET /api/Categories/{id} — Public — single object, same shape.

GET /api/Categories/{id}/brands — Public

Returns the distinct Brands that actually have Products anywhere under this Category (computed live, not stored):

```
[ { "id": 1, "name": "Samsung" }, { "id": 3, "name": "Apple" } ]
```

GET /api/SubCategories — Public

Replaces the old /api/Types.

```
[ { "id": 1, "name": "Smartphones", "categoryId": 1, "categoryName": "Electronics & Mobiles" } ]
```

GET /api/SubCategories/{id} — Public

GET /api/SubCategories/{id}/brands — Public

Same idea as Category — distinct brands with products in this subcategory.

Admin CRUD

```
POST /api/Categories    { "name": "...", "description": "..." }
```

```
PUT /api/Categories/{id}
```

```
DELETE /api/Categories/{id} // fails if it still has SubCategories — delete/reassign them first
```

```
POST /api/SubCategories  { "name": "...", "categoryId": 1 }
```

```
PUT /api/SubCategories/{id}
```

```
DELETE /api/SubCategories/{id}
```

5. Basket — fully JWT-based, Auth required on every endpoint

No query params, no basket id anywhere in the URL or body. The basket id is **always** the authenticated user's id, taken from the JWT. "Add to Cart" is the action that requires login — there is no guest basket.

GET /api/Baskets

```
{  
  "id": "b4fe71f8-...",  
  "items": [  

```

```
{ "id": 1, "productName": "...", "pictureUrl": "...", "quantity": 1, "price": 14999 }  
],  
"clientSecret": null,  
"paymentIntentId": null,  
"deliveryMethodId": null,  
"shippingPrice": null  
}
```

404 if the user has no basket yet.

POST /api/Baskets — full replace

```
{  
  "items": [  
    { "id": 1, "productName": "Samsung Galaxy A55 5G", "pictureUrl":  
      "/Images/Products/SamsungGalaxyA55.jpg", "quantity": 1, "price": 14999 }  
  ]  
}
```

Do not send an id field. This endpoint **overwrites the entire basket** — there is no per-item delete endpoint.

How to remove an item: build the new items array in the frontend without that item, then POST the whole array back. The server doesn't know "delete item X" — it only knows "this is the basket now."

DELETE /api/Baskets

Deletes the whole basket. Call this right after a successful order is created — the basket is intentionally temporary. The permanent record of a purchase is the **Order**, not the basket.

6. Checkout Flow (Basket → Payment → Order)

1. POST /api/Baskets — build the cart
2. POST /api/Payments — { deliveryMethodId } — creates/updates Stripe PaymentIntent

3. [Stripe.js] confirmCardPayment(clientSecret)

4. POST /api/Orders — { deliveryMethodId, shipToAddress } — creates the order

5. DELETE /api/Baskets — clear the cart

GET /api/Orders/DeliveryMethods — Public

```
[ { "id": 1, "shortName": "UPS1", "description": "Fastest delivery time", "deliveryTime": "1-2 Days", "price": 10 } ]
```

POST /api/Payments — Auth required

```
{ "deliveryMethodId": 1 }
```

No basket id in the URL — resolved from JWT. Response:

```
{
  "id": "...",
  "items": [...],
  "clientSecret": "pi_..._secret_...",
  "paymentIntentId": "pi_...",
  "deliveryMethodId": 1,
  "shippingPrice": 10
}
```

POST /api/Orders — Auth required

```
{
  "deliveryMethodId": 1,
  "shipToAddress": {
    "firstName": "Ahmed", "lastName": "Khaled",
    "street": "Street 1", "city": "Alex", "country": "Egypt"
  }
}
```

No basketId in the body — resolved server-side from the JWT, preventing one user from creating an order against another user's basket.

Validates stock and returns 400 (OutOfStockException) if any item exceeds available stockQuantity. On success, stock is decremented atomically with order creation.

Response: OrderToReturnDTO

```
{
  "id": "a1b2c3d4-...",
  "buyerEmail": "user@example.com",
  "orderDate": "2026-06-24T...",
  "shipToAddress": { ... },
  "deliveryMethod": "UPS1",
  "deliveryCost": 10,
  "status": "Pending",
  "items": [ { "productName": "...", "pictureUrl": "...", "price": 14999, "quantity": 1 } ],
  "subtotal": 14999,
  "total": 15009
}
```

status values: Pending, PaymentReceived, PaymentFailed, Confirmed, Shipped, Delivered, Cancelled.

GET /api/Orders — Auth required — current user's orders

GET /api/Orders/{id} — Auth required — 404 if the order isn't yours (not 403, to avoid leaking ids)

Admin

GET /api/Orders/all — every order, all users

PATCH /api/Orders/{id}/status { "status": "Shipped" }

7. Reviews

GET /api/Reviews/product/{productId} — Public

Top-level reviews with nested replies[].

```
[
  {
    "id": 1, "userDisplayName": "Ahmed", "comment": "Great!", "rating": 5,
    "createdAt": "...", "replies": []
  }
]
```

POST /api/Reviews — Auth required

```
{ "productId": 6, "comment": "Good Product", "rating": 5 }
```

One review per (user, product) — calling again updates the existing review instead of duplicating.

POST /api/Reviews/reply — Auth required

```
{ "parentReviewId": 1, "comment": "Thanks for the feedback!" }
```

PUT /api/Reviews/{id} / DELETE /api/Reviews/{id} — Auth required, owner-only (401 otherwise)

8. Admin — Customers

GET /api/Customers — all users

GET /api/Customers/{id} — id is the Identity user id (GUID string), not email

GET /api/Customers/{id}/orders

GET /api/Customers/{id}/stats — { totalOrders, totalSpend }

DELETE /api/Customers/{id}

9. Admin — Dashboard

GET /api/admin/dashboard/summary — { totalProducts, totalOrders, totalCustomers, totalRevenue }

GET /api/admin/dashboard/recent-orders?limit=10

GET /api/admin/dashboard/top-products?limit=5 — ranked by revenue

totalRevenue / top-products only count successful orders (PaymentReceived, Confirmed, Shipped, Delivered).

10. Error Responses

Standard errors (404, 401, 500, etc.):

```
{ "statusCode": 404, "errorMessage": "Product With Id = 99 Is Not Found", "errors": null }
```

Field is errorMessage, not message.

Validation errors (400, model binding/DataAnnotations failures):

```
{  
  "statusCode": 400,  
  "message": "Validation Failed",  
  "validationErrors": [  
    { "field": "Email", "errors": ["The Email field is not a valid e-mail address."] }  
  ]  
}
```

Out of stock (400):

```
{ "statusCode": 400, "errorMessage": "'Product Name' is out of stock. Available quantity: 3.",  
  "errors": null }
```

Status Meaning

200	OK
400	Validation failed / business rule violation (e.g. out of stock)
401	Missing/expired/invalid JWT
403	Authenticated but wrong role
404	Not found (including "not yours")
500	Unexpected server error

11. TypeScript Interfaces

// Auth

```
export interface UserDTO {  
  id: string;  
  email: string;  
  token: string;  
  displayName: string;  
  phoneNumber?: string;  
  userName?: string;  
  roles: string[];  
  address: AddressDTO | null;  
}
```

```
export interface AddressDTO {  
  firstName: string;  
  lastName: string;  
  street: string;  
  city: string;  
  country: string;  
}
```

```
export interface RegisterDTO {  
  email: string;  
  password: string;  
  displayName: string;  
  phoneNumber?: string;
```

```
    address?: AddressDTO; // optional
}
```

```
export interface LoginDTO {
    email: string;
    password: string;
}
```

```
// Products
```

```
export interface ProductDTO {
    id: number;
    name: string;
    description: string;
    pictureUrl: string;
    price: number;
    discount: number;
    stockQuantity: number;
    inStock: boolean;
    productBrand: string;
    subCategory: string;
    categoryId: number;
    categoryName: string;
}
```

```
export interface ProductQueryParams {
    brandId?: number;
```

```
subCategoryId?: number;
categoryId?: number;
search?: string;
sort?: 'NameAsc' | 'NameDesc' | 'PriceAsc' | 'PriceDesc';
pageNumber?: number;
pageSize?: number;
}
```

```
export interface PaginatedResult<T> {
  pageIndex: number;
  pageSize: number;
  count: number;
  data: T[];
}
```

```
export interface BrandDTO { id: number; name: string; }
```

```
export interface SubCategoryDTO {
  id: number;
  name: string;
  categoryId: number;
  categoryName: string;
}
```

```
export interface CategoryDTO {
  id: number;
```

```
name: string;
description?: string;
subCategories: SubCategoryDTO[];
}
```

// Basket

```
export interface BasketItemDTO {
  id: number;      // product id
  productName: string;
  pictureUrl: string;
  quantity: number;  // 1–100
  price: number;
}
```

```
export interface BasketDTO {
  id?: string;      // never sent by frontend
  items: BasketItemDTO[];
  clientSecret?: string;
  paymentIntentId?: string;
  deliveryMethodId?: number;
  shippingPrice?: number;
}
```

// Orders

```
export type OrderStatus =
  | 'Pending' | 'PaymentReceived' | 'PaymentFailed'
```


| 'Confirmed' | 'Shipped' | 'Delivered' | 'Cancelled';

export interface OrderItemDTO {

productName: string;

pictureUrl: string;

price: number;

quantity: number;

}

export interface DeliveryMethodDTO {

id: number;

shortName: string;

description: string;

deliveryTime: string;

price: number;

}

export interface OrderDTO {

deliveryMethodId: number;

shipToAddress: AddressDTO;

}

export interface OrderToReturnDTO {

id: string;

buyerEmail: string;

orderDate: string;

```
    shipToAddress: AddressDTO;
    deliveryMethod: string;
    deliveryCost: number;
    status: OrderStatus;
    items: OrderItemDTO[];
    subtotal: number;
    total: number;
}
```

// Reviews

```
export interface ProductReviewDTO {
    id: number;
    userDisplayName: string;
    comment: string;
    rating: number | null;
    createdAt: string;
    replies: ProductReviewDTO[];
}
```

// Errors

```
export interface ApiError {
    statusCode: number;
    errorMessage: string;
    errors?: string[];
}
```

```
export interface ValidationApiError {
  statusCode: number;
  message: string;
  validationErrors: { field: string; errors: string[] }[];
}
```

12. Quick Reference — All Endpoints

Method	Endpoint	Auth
POST	/api/Authentication/register	Public
POST	/api/Authentication/login	Public
GET	/api/Authentication/CurrentUser	Auth
GET	/api/Authentication/emailexists	Public
GET	/api/Authentication/address	Auth
PUT	/api/Authentication/address	Auth
POST	/api/Authentication/forgot-password	Public
POST	/api/Authentication/verify-reset-code	Public
POST	/api/Authentication/reset-password	Public
GET	/api/Products	Public
GET	/api/Products/{id}	Public
POST/PUT/DELETE	/api/Products[/]{id}]	Admin
GET	/api/Brands	Public
GET	/api/Brands/{id}	Public
POST/PUT/DELETE	/api/Brands[/]{id}]	Admin

Method	Endpoint	Auth
GET	/api/Categories	Public
GET	/api/Categories/{id}	Public
GET	/api/Categories/{id}/brands	Public
POST/PUT/DELETE	/api/Categories[/]{id}	Admin
GET	/api/SubCategories	Public
GET	/api/SubCategories/{id}	Public
GET	/api/SubCategories/{id}/brands	Public
POST/PUT/DELETE	/api/SubCategories[/]{id}	Admin
GET/POST/DELETE	/api/Baskets	Auth
GET	/api/Orders/DeliveryMethods	Public
POST	/api/Orders	Auth
GET	/api/Orders	Auth
GET	/api/Orders/{id}	Auth
GET	/api/Orders/all	Admin
PATCH	/api/Orders/{id}/status	Admin
POST	/api/Payments	Auth
GET	/api/Reviews/product/{productId}	Public
POST	/api/Reviews	Auth
POST	/api/Reviews/reply	Auth
PUT/DELETE	/api/Reviews/{id}	Auth (owner)
GET/DELETE	/api/Customers[/]{id}	Admin
GET	/api/Customers/{id}/orders	Admin

Method	Endpoint	Auth
GET	/api/Customers/{id}/stats	Admin
GET	/api/admin/dashboard/summary	Admin
GET	/api/admin/dashboard/recent-orders	Admin
GET	/api/admin/dashboard/top-products	Admin

Seed test accounts

Email	Password	Role
Mohamed@gmail.com	P@ssw0rd	Admin
Salma@gmail.com	P@ssw0rd	SuperAdmin